

JavaScript Debugging Notes — Portfolio Template Session

A collection of real mistakes made while building navbar active-link tracking, theme switching, portfolio filters, and a testimonials carousel — with the underlying concept behind each one. Read this before starting a new vanilla-JS UI project.

1. Always check the CSS before writing the JS

The mistake: Assuming that adding/removing a class called `active` will always produce a visible change.

What actually happened: In this project, `.nav-links a.active` and `.group-[.active]:opacity-100` (font picker checkmark) had real CSS rules behind them — toggling `active` worked immediately. But `.portfolio-filter` and `.carousel-indicator` had **no CSS rule for `.active` at all** — their “selected” look came from hardcoded Tailwind utility classes baked directly into the HTML (e.g. `bg-linear-to-r from-primary to-secondary`, `bg-accent`, `border-primary`). Toggling `active` there did nothing visually, even though the JS logic was correct.

The concept: `grep`/search your CSS file for the class name *before* writing the toggle logic. If there’s no matching rule, you need to swap the actual utility classes yourself (remove the “unselected” set, add the “selected” set) instead of relying on one class name to do the work.

2. `querySelector` VS `querySelectorAll`

The mistake:

```
var portfolioCards = document.querySelector(".portfolio-item"); //  
single element!
```

Looping with `portfolioCards[i]` and `portfolioCards.length` silently does nothing — `.length` is `undefined`, so `i < undefined` is `false` from the first iteration.

The concept: `querySelector` returns the **first match only** (a single element). `querySelectorAll` returns a **`NodeList`** (needs indexing and has a real `.length`). If you intend to loop, you almost always want `querySelectorAll`.

3. Looping bugs: comparing against the collection, not `.length`

The mistake:

```
for (var i = 0; i < portfolioButtons; i++) { ... } // ✗ compares i  
to the NodeList itself
```

A NodeList converts to NaN when compared to a number, so `i < NaN` is always false — the loop body never runs, even once.

The concept: Always compare against `.length`:

```
for (var i = 0; i < portfolioButtons.length; i++) { ... }
```

4. var vs let inside loops with event listeners (classic closure bug)

The mistake:

```
for (var i = 0; i < carouselIndicator.length; i++) {  
  carouselIndicator[i].addEventListener("click", function () {  
    currentIndex = i; // always uses the FINAL value of i  
  });  
}
```

The concept: `var` is function-scoped — there is only **one shared i** for the entire loop. By the time any click actually happens, the loop has already finished, so every listener reads the same final value of `i` (usually `array.length`, which is out of bounds). `let` is block-scoped — it creates a **new independent i per iteration**, so each listener correctly remembers its own value. Fix: change `var i` to `let i` in loops that create closures (event listeners, timeouts, etc).

5. State must live outside the event handler to persist

The mistake:

```
nextTestimonial.addEventListener("click", function () {  
  var currentIndex = 0; // ✗ resets to 0 on every click  
  ...  
});
```

The concept: A variable declared *inside* a function is recreated fresh every time that function runs. If a value needs to persist across multiple clicks (like a carousel index, or a running total), declare it **outside** the event listener, at the top level of your script.

Good structure for stateful UI components: state variable → one render/update function that reads the state and touches the DOM → small event handlers that only change the state and then call the render function.

6. getAttribute always returns a string (or null) — never a boolean

The mistake:

```
if (toggleThemButton.getAttribute("aria-pressed") === true) { ... }  
// ✗ never true
```

The concept: HTML attributes are text. `getAttribute("aria-pressed")` returns `"true"` or `"false"` as **strings**, never the boolean `true/false`. Compare against the string: `=== "true"`. Also remember `getAttribute`

returns null if the attribute isn't set at all in the HTML — make sure your default state is actually written into the markup, or handle null explicitly.

7. | is bitwise OR, not “or this string or that string”

The mistake:

```
localStorage.setItem('theme', 'dark' | 'light') // always stores "0"
```

The concept: | between two strings forces both to numbers first — both 'dark' and 'light' become NaN, and NaN | NaN evaluates to 0. This line always saved the string "0", regardless of the actual theme. Just store the variable you already computed (theme), not a | expression.

8. classList is spelled with a capital “L”

The mistake: element.classList.add(...) — lowercase l.

The concept: JavaScript is case-sensitive. classList doesn't exist; classList (camelCase) does. Typos like this throw a runtime error that stops the rest of the script block from running.

9. A stray semicolon after if (...) creates an empty statement

The mistake:

```
if (sectionID.contains(linkHref)); // ✗ semicolon ends the if here
links.classList.add(active);      // runs unconditionally, every
time
```

The concept: if (condition); is valid JS — it's an if whose entire body is “do nothing.” The line right after it is **not** inside the if anymore; it just runs regardless of the condition. Always double check that { } wraps the intended body when the condition matters.

10. Class names passed to classList.add() need quotes

The mistake: element.classList.add(active) — active here is interpreted as a variable name, not the string "active". If no such variable exists, this throws a ReferenceError.

The concept: classList.add("active") — always a string.

11. Event delegation + closest() for reliable click targets

The concept: When listening for clicks on a container (#portfolio-filters, #portfolio-grid, etc.) instead of on each individual button, e.target is whatever the user *actually* clicked — which might be an

icon or span nested *inside* the button, not the button itself. Use `e.target.closest(".portfolio-filter")` to reliably get the intended element regardless of what was clicked inside it, and treat a null result as “the click wasn’t on a relevant element at all.”

12. Avoiding “flash of wrong theme” on page load

The concept: - Code that changes what’s *visually painted* (adding/removing the dark class on `<html>`) must run **before** the browser paints — put it in a small **inline** `<script>` as early as possible in `<head>`, before your CSS `<link>`. - That early script can only safely touch elements that are guaranteed to exist already — `document.documentElement` (the `<html>` tag) always exists, but a button like `#theme-toggle-button` (defined later in `<body>`) does **not** exist yet at that point in parsing. Calling `document.getElementById(...)` for it returns `null`, and calling any method on `null` throws a fatal error that stops the rest of that script block. - Split the logic: **visual class toggle** → early inline script in `<head>`. **Anything touching a specific element like a button** (e.g. syncing `aria-pressed`) → your regular script, later in the page, after that element actually exists in the DOM.

13. `localStorage.getItem()` can return null

The concept: If nothing has been saved yet, `getItem` returns `null`, not a default string like `"light"`. Always handle the “nothing saved yet” case explicitly and decide what the fallback/default should be.

General checklist for next time

- ☐ Search the CSS file for the class name before relying on it to change the look of something.
- ☐ `querySelector` = one element. `querySelectorAll` = a list, needs indexing and `.length`.
- ☐ Loop conditions compare against `.length`, never the collection itself.
- ☐ Use `let` (not `var`) in any loop that creates event listeners/closures.
- ☐ State that must survive across multiple events lives outside the event handler, not inside it.
- ☐ `getAttribute` returns strings/null — never compare it to a boolean.
- ☐ Double-check operators: `|` is bitwise, not “or this string or that string” — use `||` for logic, or just store the right variable.
- ☐ `classList` — capital L.
- ☐ Watch for stray semicolons right after `if (...)`.
- ☐ Wrap string class names in quotes when passing to `.add()/remove()`.
- ☐ Prefer event delegation + `.closest()` over attaching a listener to every single item.
- ☐ Anything that must apply before first paint (theme class) goes in an early inline `<head>` script and must not reference elements defined later in `<body>`.